

Pulse — Instalación y explicación del proyecto

Prueba Técnica — Desarrollador Full Stack · Daniel Rodríguez

Red social con arquitectura de microservicios: autenticación JWT, perfil de usuario, publicaciones y likes en tiempo real (WebSocket/STOMP).

1. Tecnologías

Capa	Tecnología
Backend	Java 21, Spring Boot 3.3 (Web, Security, Data JPA, WebSocket, Actuator, Validation)
Base de datos	PostgreSQL 16, Flyway (migraciones), PL/pgSQL (2 PROCEDURES + 1 FUNCTION)
Frontend	Angular 19 (standalone), NgRx SignalStore, @stomp/rx-stomp, TypeScript estricto
Infraestructura	Docker (multi-stage), Docker Compose, nginx
Documentación	Swagger / OpenAPI (springdoc) en /docs de cada servicio
Pruebas	JUnit 5, Mockito, MockMvc, Testcontainers (PostgreSQL real)

2. Arquitectura

```
Navegador ■■■ frontend (nginx :4200)
          ■■■ /auth, /users ■■■ auth-service :8081 ■■■ schema "auth"
          ■■■ /posts       ■■■ posts-service :8082 ■■■ schema "posts"
          ■■■ /ws (STOMP)   ■■■ posts-service :8082
                           PostgreSQL 16 (socialdb)
```

- **auth-service**: login JWT (POST recomendado + GET literal del enunciado) y perfil (GET /users/me). Dueño del schema **auth**.
- **posts-service**: publicaciones, likes mediante **stored procedures PL/pgSQL** y difusión en tiempo real por WebSocket al tópico /topic/likes. Dueño del schema **posts**.
- **frontend**: SPA Angular servida por nginx; nginx hace proxy de API y WebSocket (un solo origen — sin CORS). Estado global con **NgRx SignalStore** (singleton + signals).

- Los servicios **no se llaman entre sí**: el JWT (HS256, secreto compartido) lleva id, username y alias; el alias del autor se denormaliza en el post.

3. Requisitos previos

Solo se necesita:

- **Docker Desktop** (Windows/macOS) o Docker Engine + Compose v2 (Linux).
- Puertos libres: **4200**, **8081**, **8082**, **5432**.

Para desarrollo local adicional (opcional): JDK 21, Maven 3.9, Node 20+.

4. Instalación y ejecución

```
# 1. Clonar el repositorio
git clone <URL-DEL-REPOSITORIO>
cd periferia-social

# 2. Levantar todo el stack (primera vez: ~3-6 min)
docker compose up --build
```

Docker Compose levanta, en orden y con healthchecks: PostgreSQL → auth-service y posts-service (cada uno ejecuta sus migraciones Flyway y su seeder) → frontend.

Verificación rápida:

Qué	URL	Esperado
Aplicación	http://localhost:4200	Pantalla de login de Pulse
Swagger auth	http://localhost:8081/docs	UI de OpenAPI
Swagger posts	http://localhost:8082/docs	UI de OpenAPI
Salud auth	http://localhost:8081/actuator/health	{"status": "UP"}
Salud posts	http://localhost:8082/actuator/health	{"status": "UP"}

Apagar: **docker compose down** · **Reset total (borra BD):** **docker compose down -v**

5. Usuarios de prueba

El seeder crea 5 usuarios (contraseñas Bcrypt) y una publicación por usuario. El script independiente equivalente está en **db/seed.sql**.

Usuario	Clave	Alias
mariana	Pulse2026!	@marilo
carlos	Pulse2026!	@cgomez
valentina	Pulse2026!	@valen
andres	Pulse2026!	@atorres
daniela	Pulse2026!	@danim

6. Uso de la aplicación

1. **Login** — entra a `http://localhost:4200`; usa un chip de usuario demo (se llenan las credenciales) e Ingresar. El backend responde un JWT que el frontend guarda y adjunta vía interceptor.
2. **Publicaciones** — feed con las publicaciones de **otros** usuarios (las propias no aparecen, como pide el enunciado), con foto del autor, imagen de la publicación (si tiene), total de likes y corazón para dar/quitar like.
3. **Tiempo real** — abre una segunda ventana (incógnito) con otro usuario: los **likes** actualizan el contador en ambas ventanas al instante, y las **publicaciones nuevas** aparecen solas en el feed del otro usuario, sin recargar (WebSocket STOMP → SignalStore → señal → render).
4. **Crear publicación** — pestaña "Crear": mensaje y foto opcional (JPEG/PNG/WebP hasta 2MB); la fecha la asigna el servidor al guardar.
5. **Perfil** — clic en tu usuario (arriba a la derecha): nombres, apellidos, fecha de nacimiento y alias desde `GET /users/me`. Desde ahí puedes **editar tus nombres** y **subir tu foto de perfil** (clic sobre el avatar).

7. Stored procedures (requisito de BD)

Definidas en la migración `V2__create_procedures.sql` de posts-service e invocadas desde Java:

Rutina	Tipo	Rol
<code>sp_register_like(post, user, INOUT total)</code>	PROCEDURE	Inserta el like (idempotente: <code>ON CONFLICT DO NOTHING + PK</code> compuesta) y retorna el nuevo total. Se invoca con <code>CALL</code> vía <code>CallableStatement</code> .
<code>sp_remove_like(post, user, INOUT total)</code>	PROCEDURE	Elimina el like y retorna el nuevo total.

<code>sp_get_posts_with_likes(current_user_id)</code>	FUNCTION RETURNS TABLE	Feed: posts de otros usuarios + conteo de likes + <code>liked_by_me</code> .
---	------------------------	--

PostgreSQL distingue PROCEDURE (se invoca con `CALL`, admite `INOUT`) de FUNCTION (retorna filas). Las mutaciones son PROCEDURES reales y la lectura es una FUNCTION — cada rutina en su forma idiomática. El datasource usa `escapeSyntaxCallMode=callIfNoReturn` para que el driver emita `CALL`.

8. Pruebas

Desde `backend/auth-service` o `backend/posts-service`:

```
mvn test      # unitarias (JUnit 5 + Mockito) — no requieren Docker
mvn verify    # + integración (Testcontainers con PostgreSQL real)
```

Resultados actuales: **auth-service 11 unitarias + 12 integración · posts-service 7 unitarias + 16 integración · frontend 18 specs (Karma/Jasmine)** — incluyen la ejecución real de las stored procedures (idempotencia verificada contra la BD), acumulación de likes entre usuarios distintos, el ciclo completo de imágenes y clientes STOMP reales que reciben los broadcasts de likes y publicaciones.

9. Estructura del repositorio

```
■■■■ docker-compose.yml      # PostgreSQL + auth + posts + frontend
■■■■ db/seed.sql             # usuarios y posts predefinidos (entregable)
■■■■ backend/
■    ■■■■ auth-service/      # JWT + perfil · schema auth · Dockerfile
■    ■■■■ posts-service/    # posts + likes + WS · schema posts · Dockerfile
■■■■ frontend/              # Angular 19 + SignalStore + nginx · Dockerfile
■■■■ README.md              # arquitectura y decisiones técnicas (detallado)
```

10. Solución de problemas

Problema	Causa/solución
Un puerto está ocupado	Libera 4200/8081/8082/5432 o ajusta el mapeo en <code>docker-compose.yml</code> .
<code>docker compose up</code> falla al descargar	Reintenta (dependencias de Maven/npm se cachean en capas).
El contador no se mueve en tiempo real	Verifica el punto verde junto al logo (WebSocket conectado); revisa <code>docker logs pulse-posts-service</code> .

Quiero datos limpios	<code>docker compose down -v && docker compose up --build.</code>
----------------------	---